Chapter 6 解答

CH06-Q01

1. Logical sector payload capacity：

$$10{,}000 \times 4 \times 500 \times 512 = 10{,}240{,}000{,}000 \text{ bytes} \approx 10.24 \text{ GB}$$

以 2^{30} bytes/GiB：

$$10{,}240{,}000{,}000/2^{30} \approx 9.5367 \text{ GiB}.$$

2. 12,000 RPM 的 rotation period：

$$60/12000 = 0.005 \text{ s} = 5 \text{ ms}.$$

Average rotational latency 是半圈 2.5 ms。

3. 一 track 有 500 sectors，單 sector 的 rotation-time share：

$$5 \text{ ms}/500 = 0.010 \text{ ms}.$$

4. Random single-sector average latency：

$$4.5 + 2.5 + 0.010 + 0.10 = 7.110 \text{ ms}.$$

同 cylinder 的後續 99 sectors 不用再 seek，通常也只需第一次 rotating to start；若連續 sector 讀取，其 transfer 為

$$100 \times 0.010 = 1.0 \text{ ms}.$$

第一次 random read 加上後續 99 sectors 的簡化連續request模型 是

$$7.110 + 0.99 = 8.100 \text{ ms}.$$

Seek 與 initial rotational positioning 被 100 sectors 的 sequential request amortize。真機須加 track crossing、gap 與 controller protocol。

CH06-Q02

1. Availability：

$$A = \frac{100000}{100000 + 4} \approx 0.9999600016.$$

2. Expected downtime proportion：

$$1 - A \approx 3.99984 \times 10^{-5}.$$

在 8,760 hours 的 expected downtime：

$$8760(3.99984 \times 10^{-5}) \approx 0.350386 \text{ h} \approx 21.02 \text{ min}.$$

3. MTTR=0.5 hour 時：

$$A' = \frac{100000}{100000.5} \approx 0.9999950000,$$

Chapter 6 解答 （續）

expected downtime 約：

$$8760(1 - A') \approx 0.0438 \text{ h} \approx 2.63 \text{ min}.$$

新expected downtime比原本少約 $21.02 - 2.63 = 18.39$ minutes/year。

4. 不足以推出每年最多停一次。MTTF 是平均值，不描述 failure 的 variance、interarrival distribution、correlated outages、maintenance window 與 observation window。要預測次數需要 failure model 與實際 incident data；要預測停機時長，還要 repair distribution。

CH06-Q03

1. Write amplification：

$$WA = 384/120 = 3.2.$$

2. 粗略 total host writes：

$$\frac{400 \text{ GB} \times 3000}{3.2} = 375{,}000 \text{ GB} = 375 \text{ TB}.$$

這是模型值，不是 warranty 或 lifetime guarantee。

3. physical NAND仍是400 GB，保留20% spare後host可用320 GB。P/E budget的分子仍用400 GB，不把host容量再扣一次。給定另一工作集下WA=4：

$$\frac{400 \text{ GB} \times 3000}{4} = 300{,}000 \text{ GB} = 300 \text{ TB}.$$

可得 host endurance 約下降 20%。這不表示增加spare導致WA升高；一般spare有助GC，但此題同時改變working set且直接給定觀察到的WA。實機endurance仍需controller與媒體資料。

4. Drive 快滿時，GC 可用的空白 erase blocks 少，必須搬移更多 valid pages 才能 reclaim block；random small writes 增加 partial-page copy；mapping/overprovisioning 也影響。Benchmark 應在指定 free-space ratio、random/sequential mix、queue depth 與 warm-up 後量測 host bytes、NAND bytes、GC、tail latency 與 endurance，而非只量 peak bandwidth。

CH06-Q04

1. 每秒 overhead cycles：

- Polling：$20{,}000 \times 100 = 2.0 \times 10^6$ cycles/s。
- Interrupt：$20{,}000 \times 1500 = 3.0 \times 10^7$ cycles/s。

2. 處理器利用率：

Chapter 6 解答 （續）

- Polling：$2.0 \times 10^6 / (3.0 \times 10^9) = 0.000667 \approx 0.067\%$。
- Interrupt：$3.0 \times 10^7 / (3.0 \times 10^9) = 0.01 = 1.0\%$。

在題設「一次 poll 立即完成」下，polling 的 CPU overhead 反而較低。

3. 嚴格依本題「每I/O恰好一次poll、沒有額外busy wait」的假設，polling overhead在沒有其他工作或有100個runnable threads時都較低，不能僅由thread數改判interrupt較佳。若改成反覆poll至completion，才需另外計等待cycles、scheduler delay、power與機會成本；本題未給這些數字。
4. Hybrid 可在提交 I/O 後短暫 spin，因為 completion 可能很快；超過門檻後進入 interrupt/sleep，並用 interrupt coalescing 或 completion queue。策略要依 device latency distribution、queue depth 與 CPU opportunity cost 量測，而非固定用一個方法。

CH06-Q05

1. 64 MiB：

$$64 \times 2^{20} = 67{,}108{,}864 \text{ bytes}.$$

Payload 頻寬：

$$67{,}108{,}864 / 0.020 = 3{,}355{,}443{,}200 \text{ B/s} \approx 3.355 \text{ GB/s}.$$

2. DMA setup：

$$500 / (3 \times 10^9) = 1.6667 \times 10^{-7} \text{ s} \approx 0.167 \ \mu s.$$

3. 每 transaction 16 bytes，每4 bytes需4 cycles，所以16-byte transaction的搬移成本是16 cycles，另加10 cycles，共26。Transactions 數：

$$67{,}108{,}864 / 16 = 4{,}194{,}304.$$

CPU cycles：

$$4{,}194{,}304 \times 26 = 109{,}051{,}904.$$

約 36.35 ms CPU execution at 3 GHz，尚不含 cache miss、device wait 與 OS overhead。

4. DMA 把資料搬移從 core instruction stream 移到 controller，但 memory-to-device traffic 仍占 memory/interconnect bandwidth。正確性上需處理 DMA address translation/IOMMU、cache coherence 或 explicit cache maintenance、buffer lifetime、completion ordering、error/timeout 與 write visibility。

Chapter 6 解答 （續）

CH06-Q06

1. Devices合計上限是6×80000=480000 IOPS。
2. 同一條request path的sustained上限不超過各資源容量的最小值：min(480000,120000,300000,150000)=120000 IOPS，瓶頸為HBA。這只是第一個容量上界，未計排隊、共享協定與服務時間分布。
3. 增加兩顆SSD後device上限640000，但HBA仍120000，原上限不變。
4. 以下是三個**各自從原配置出發**的明定升級模型，不假設調整一個非瓶頸就能自動提高IOPS：

配置方案與明定新容量	Devices	HBA合計	Memory	Driver/CPU	新容量上界及瓶頸
更換HBA至200000，其餘不變	480000	200000	300000	150000	150000，Driver/CPU
兩個獨立120000 HBA各接3顆SSD，並配置兩條獨立150000 driver/CPU路徑；工作可均分且無額外共享限制	480000	240000	300000	300000	240000，兩HBA
HBA與driver/CPU均升至500000，其餘不變	480000	500000	300000	500000	300000，Memory

每個數值都是本方案的假設容量，用以反推下一個瓶頸，不是未實測的硬體效能承諾。只降低driver overhead而保留原HBA，仍被120000限制，不能當成已提高原上限。Batching可減少per-I/O CPU成本，但未同時提高HBA容量也沒有突破120000；增大request size改變bytes/s與工作負載，不能直接當成同一IOPS指標的提高。

CH06-Q07

1. Nominal usable capacities：

- RAID 0：$8 \times 8 = 64$ TB。
- RAID 5：$(8 - 1) \times 8 = 56$ TB。
- RAID 6：$(8 - 2) \times 8 = 48$ TB。
- RAID 10：4 mirrors × 8 TB = 32 TB。

2. Failure model：

Chapter 6 解答 （續）

- RAID 0：任一 disk failure 即失去 array 的完整資料。
- RAID 5：單一 disk failure 可由 parity 重建。
- RAID 6：在雙 check-symbol 模型內可容忍兩顆 failures。
- RAID 10：每組 mirror 可失去一顆；不同 mirror 同時各失一顆也可能繼續工作。任一 pair 雙失才使該 stripe 失去資料。

3. 題中簡化 read-modify-write：read old data、read old parity、write new data、write new parity，共 4 disk accesses；其中 2 reads、2 writes。
4. 更新 data、P、Q 通常要 read old data、old P、old Q，再 write data、new P、new Q，共 6 accesses；實際 controller/cache/protocol 可不同。

CH06-Q08

1. 本題指定只重建每顆disk的9 TB有效區域。RAID5重建一個遺失block需讀其餘11顆各一個block，因此surviving reads為

$$11 \times 9\ \mathrm{TB} = 99\ \mathrm{TB}.$$

這是讀取流量，不是replacement disk容量。array的99 TB有效logical資料也不是全寫回單一replacement disk。

2. Replacement只寫入9 TB，所以aggregate traffic為

$$99 + 9 = 108\ \mathrm{TB}.$$

共用aggregate 1 GB/s路徑時，lower bound為

$$108 \times 10^{12}/10^{9} = 108000\ \mathrm{s} = 30\ \mathrm{h}.$$

不辨識有效區域而重建整顆10 TB時，要讀110 TB、寫10 TB，合計120 TB，lower bound為120000 s，約33.33 h。若「1 GB/s rebuild rate」另定義為replacement輸出速率，9 TB/1 GB/s才是9000 s=2.5 h，且surviving-read路徑需同時支援11 GB/s；兩種頻寬定義不能混用。

3. RAID6的重建讀取集合取決於失效disk種類、同時失效數量、P/Q恢復演算法與controller對有效stripe的追蹤。可略過一個check block的實作與讀全部survivors的實作，其流量不同，不能僅按10/11容量比推定時間，也不能宣稱RAID6單失效必然比RAID5讀更多。
4. 故障暴露機率取決於重建時間與剩餘disk的hazard、latent media errors及power/cooling/controller等相關失效。單一MTTF只給平均值，不提供時間變化或相關性；本題traffic下限不能代替可靠度模型。

CH06-Q09

1. 64 MiB：

Chapter 6 解答 （續）

$$64 \times 2^{20} \times 8 = 536{,}870{,}912 \text{ bits}.$$

2. 10 Gbit/s × 80%：

$$8 \text{ Gbit/s} = 1 \text{ GB/s}$$

有效 payload bandwidth 約 10^9 decimal bytes/s，或 0.9313 GiB/s。

3. Transfer：

$$67{,}108{,}864/10^9 = 0.067108864 \text{ s}.$$

4 MiB batches 有 $64/4 = 16$ batches；第一次 transfer 前可能有一次 setup，批次之間有 15 次；若兩端都已建立 connection，只算 15 inter-batch gaps：

$$15 \times 0.20 \text{ ms} = 3.0 \text{ ms}.$$

總時間為 $67.108864 + 3.0 = 70.108864$ ms。若每次 transaction 另需首尾 RTT，則加 0.2 ms，變成 70.308864 ms；假設必須明寫。

4. 不可以。100 streams 可以提高 concurrency，但瓶頸仍受 link bandwidth，且 queueing 增加 latency與 tail；同時有 protocol/CPU/storage backend limit。Latency 與 bandwidth 是不同限制，pipelining 可隱藏部分 latency，不能創造超過 link capacity 的資料率。

CH06-Q10

1. μ=1/0.004=250/s，ρ=200/250=0.8，W=1/(250−200)=0.020 s，Wq=20−4=16 ms，L=λW=4。Payload頻寬200×4096=819200 B/s，不把queue內的4筆再乘進吞吐量。
2. 1/(250−λ)<=0.010，分母為正，得到250−λ>=100，故λ<=150/s。等號時ρ=0.6、W=10 ms、L=150×0.010=1.5；平均數不需為整數。這是平均response限制，未承諾99th percentile也不超過10 ms。
3. λ從200升到240/s時，service仍4 ms，W卻從20升至100 ms，L從4升至24。Little定律只需適當穩態及有限平均，不要求Poisson；W=1/(μ−λ)才依賴本題M/M/1模型。兩個式子的假設不能混在一起。

CH06-Q11

1. 1010 XOR 1100=0110，再XOR0110為0000，再XOR1111得到P=1111。更新後重算 1010 XOR 0101 XOR 0110 XOR 1111=0110。Shortcut是1111 XOR 1100 XOR 0101=0110，結果相同，因old D1在舊parity中出現一次，另XOR一次正好消去。
2. m=4。RMW讀k個old data與old parity，共k+1讀；寫k個new data與new parity，共k+1寫，總2k+2。Reconstruct讀m−k個未改data，再寫k個new data與parity，共m+1=5。

Chapter 6 解答 （續）

k	RMW讀/寫	RMW bytes	Reconstruct讀/寫	Reconstruct bytes
1	2/2	16384	3/2	20480
3	4/4	32768	1/4	20480

此表是總工作量，不能把總操作數直接當作並行elapsed time。Cache命中、同stripe lock及device placement會改變效能。 3. D1已是0101時，全部data的XOR應0110，舊P仍1111，故P=XOR(data)失效。若此時另失D0，用舊parity恢復會得到錯值。再存一份相同parity仍可能兩份都舊，無法保證data/parity屬同一交易。需受保護的重作資訊或其他具明定持久順序的原子更新協定。

CH06-Q12

1. P=5 XOR 7 XOR 9=11。域內2・7=14；2・9先左移得到binary10010，截低4 bits為0010，再XOR0011得0001，所以2・9=1，4・9=2。Q=5 XOR 14 XOR 2=9。
2. 已知D1=7，X=P XOR 7=12；Y=Q XOR(2・7)=9 XOR 14=7。未知方程是D0+D2=12、D0+4D2=7，兩式XOR得5D2=7 XOR 12=11。用域乘法，5・11=(4+1)・11=4・11 XOR 11。2・11=5、4・11=10，所以5・11=10 XOR 11=1，核對逆元素。D2=11・11=9，D0=12 XOR 9=5。驗回5 XOR 7 XOR 9=11及5 XOR 14 XOR 2=9。
3. 本題明知哪兩個symbols缺失，只有其值未知；silent corruption可能不知道哪個位置錯，方程的未知數還包括錯誤位置。兩個erasure可恢復不能直接推出可定位並修正任意兩個未知錯誤。

CH06-Q13

1. 讀7×6=42 TB，寫6 TB，合計48 TB。共用2 GB/s下，T=48×10^12/(2×10^9)=24000 s=20/3小時，約6.6667小時。
2. 其餘7顆皆存活的機率為exp(−7λT)，故至少一顆再失效的機率為1−exp(−7×10^−5×20/3)，約0.000466558，即0.04666%。若實際rebuild更久，固定hazard模型下此機率更大。
3. 若2 GB/s是replacement輸出，T=6×10^12/(2×10^9)=3000 s=5/6小時，但surviving讀取需14 GB/s，總路徑需16 GB/s。第2小題已條件化於第一顆失效且只觀察一次rebuild；年風險還需第一顆失效的發生率、修復週期、相關失效與其他錯誤模式，不能由此值直接替代。

Chapter 6 解答 （續）

原創驗算程式附錄: storage_math.py

```
"""Re-runnable storage and I/O calculations used in Chapter 6."""

from dataclasses import dataclass


def average_rotation_ms(rpm: float) -> float:
    return (60.0 / rpm) * 1000.0 / 2.0


def availability(mttf_hours: float, mttr_hours: float) -> float:
    return mttf_hours / (mttf_hours + mttr_hours)


def raid_capacity_tb(level: int, disks: int, disk_tb: float) -> float:
    if level == 0:
        data_disks = disks
    elif level == 1:
        data_disks = disks / 2
    elif level == 5:
        data_disks = disks - 1
    elif level == 6:
        data_disks = disks - 2
    else:
        raise ValueError(f"unsupported level: {level}")
    return data_disks * disk_tb


@dataclass(frozen=True)
class IOPath:
    device_iops: int
    controller_iops: int
    link_iops: int
    driver_iops: int

    def bottleneck(self) -> tuple[str, int]:
        limits = {
            "device": self.device_iops,
            "controller": self.controller_iops,
            "link": self.link_iops,
            "driver": self.driver_iops,
        }
        name = min(limits, key=limits.get)
        return name, limits[name]
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 6 解答 （續）

```
def main() -> None:
    rotation = average_rotation_ms(12000)
    assert abs(rotation - 2.5) < 1e-12
    one_sector_transfer_ms = (60.0 / 12000) * 1000.0 / 500
    assert abs(one_sector_transfer_ms - 0.01) < 1e-12
    access_ms = 4.5 + rotation + one_sector_transfer_ms + 0.10
    assert abs(access_ms - 7.11) < 1e-12

    avail = availability(100000, 4)
    downtime_minutes = (1 - avail) * 8760 * 60
    assert abs(downtime_minutes - 21.0223) < 0.001

    capacities = {
        "RAID0": raid_capacity_tb(0, 8, 8),
        "RAID5": raid_capacity_tb(5, 8, 8),
        "RAID6": raid_capacity_tb(6, 8, 8),
        "RAID10": raid_capacity_tb(1, 8, 8),
    }
    assert capacities == {"RAID0": 64, "RAID5": 56, "RAID6": 48, "RAID10": 32}

    path = IOPath(device_iops=480000, controller_iops=120000, link_iops=300000,
driver_iops=150000)
    assert path.bottleneck() == ("controller", 120000)

    transfer_bytes = 64 * 2**20
    dma_bandwidth = transfer_bytes / 0.020
    assert abs(dma_bandwidth - 3.3554432e9) < 1.0
    assert abs(500 / 3e9 - 1.6666666666666667e-7) < 1e-20

    print(f"average rotation latency: {rotation:.3f} ms")
    print(f"random sector read model: {access_ms:.3f} ms")
    print(f"availability: {avail:.9f}")
    print(f"expected downtime: {downtime_minutes:.3f} min/year")
    print("RAID capacity:", ", ".join(f"{name}={value} TB" for name, value in
capacities.items()))
    print(f"I/O bottleneck: {path.bottleneck()[0]} at {path.bottleneck()[1]:,} IOPS")
    print(f"DMA payload bandwidth: {dma_bandwidth / 1e9:.3f} GB/s")
    print("storage checks: PASS")


if __name__ == "__main__":
    main()
```

參考資料與署名

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study